

Why Learning to Code Properly Still Matters in the Age of AI

What does our experience buy us now?

Several thousand lines of responsibility.

John Azariah

The payment succeeded. What failed?

A familiar workflow. An ambiguous outcome.

```
var stock = await inventory.CheckStock(request.Items);  
if (!stock.IsAvailable)  
    return OrderResult.Failed("Out of stock");  
  
var price = pricing.Calculate(request.Items, request.Coupon);  
var charge = await payment.Charge(request.PaymentMethod, price.Total);  
if (!charge.Succeeded)  
    return OrderResult.Failed("Payment failed");  
  
await inventory.Reserve(request.Items);  
await email.SendConfirmation(request.Customer, price);  
return OrderResult.Success(charge.TransactionId);
```

A timeout is not proof that the payment failed.

What experience notices

Move knowledge out of a review comment and into the system.

Notice	Establish	Make it survive
A timeout may hide a charge	Provider retry and reconciliation behaviour	Adapter contract + integration checks
Stock changes after a lookup	Reservation and consistency policy	Workflow rules + failure-path tests
Confirmation can fail separately	Whether the order stays successful	Explicit outcome + delivery policy

Experience is useful when the next change can inherit it.

A brief frog

One program. Different interpretations.

```
let adventure = frog {  
  jump  
  croak  
  jump  
  eat_fly  
}
```

DSL sketch; builder omitted.

Narrative

Describe the actions.

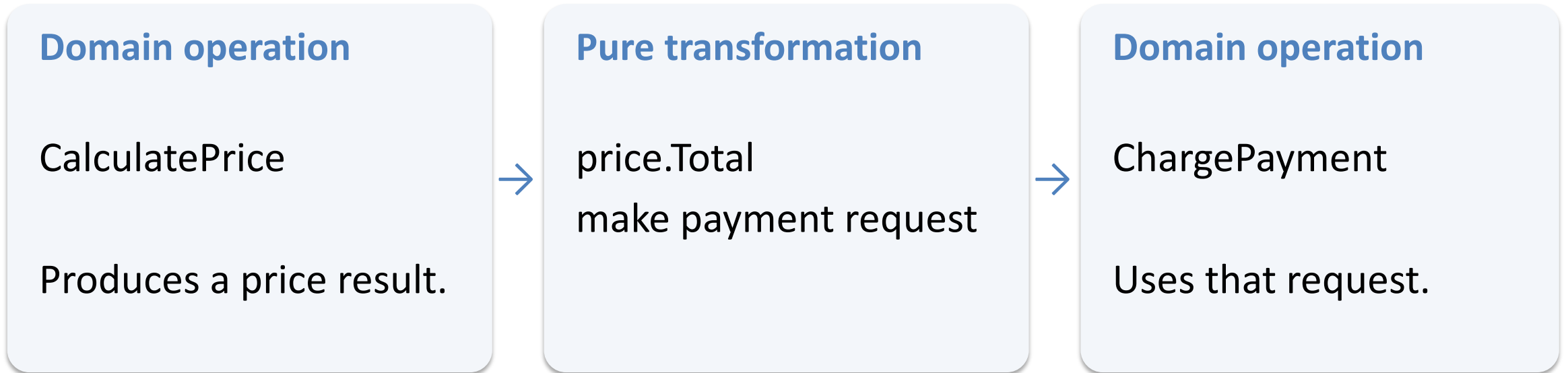
Simulation

Update the frog's state.

The interpreter supplies meaning—and how results combine.

The frog doesn't have a payment gateway

Now an operation needs a value produced by an earlier operation.



Sequencing makes the result available to the next computation.

Domain vocabulary is not the sequencing machinery.

Write the decision once

$K\langle F, A \rangle$: an A produced in interpretation F

```
public static K<F, string> PlaceOrder<F>(string items)
  where F : OrderAlgebra<F> =>
  from stock in F.CheckStock(items)
  from price in F.CalculatePrice(items)
  from charge in F.ChargePayment(price)
  select charge;
```

Id

Return a value.

Trace

Value + operation log.

Small composition example—not a production order system.

Two representations. What survives the journey?

Explicit program syntax

Instructions and the structure connecting them.



Program through interpreters

One uniform program;
different interpretations.

Condition: uniform behaviour across interpretations.

Recovery may preserve meaning without preserving exact syntax.

Preserving the program does not prove the program right.

My explanation needed debugging too

The later work sharpened what the original account could claim.

“It compiles”

Some properties
were checked.

“These tests pass”

These cases
were exercised.

“This is preserved”

A stated property
has evidence.

“This fits our domain”

The requirement
still needs
establishing.

Separate claims—not automatic implications.

What exactly have we established?

May we remove the second read?

Same arguments do not necessarily mean the same observation.

Before

```
first = ReadStock(item)
...
second = ReadStock(item)
use(first, second)
```

Proposed change

```
first = ReadStock(item)
...
second = first
use(first, second)
```

Live service

The second result may differ.
The rewrite can change behaviour.

Fixed snapshot

Reuse may be justified—subject to the observable contract.

The operation name didn't tell us. The service contract did.

Give the agent something worth preserving

Work brief	Make the condition explicit
Goal	Reduce repeated stock lookups within one order evaluation.
Boundary	Reuse only inside one immutable snapshot. No payment-path changes.
Evidence	Live-read counterexample; same-snapshot reuse; cross-snapshot separation.
Unknowns	Identify any service guarantee the repository cannot establish.

Keep the important conditions in the system, not just the chat.

Delegate according to consequence

Vibe coding and structured workflows are modes—not rival identities.

Bounded exploration

Synthetic data
No production connection
Easy to discard

Consequential change

Explicit acceptance conditions
Protected execution
Review and recovery

“Temporary” is not a risk classification.

Choose autonomy by consequence and recoverability.

Keep learning where the mistakes are expensive

Understand behaviour

Read unfamiliar code.
Follow state and effects.
Investigate failures.

Know what checks establish

Predict before running.
Seek counterexamples.
Read past the explanation.

Learn the domain

What is an accepted order?
What does “cancel” mean?

Preserve the apprenticeship

Give bounded ownership.
Keep the feedback from consequences.

What can I now recognise, express, or check that I couldn't before?

What your experience buys

AI makes implementation cheaper.

**Understanding changes
what we can trust.**

The code may arrive in seconds.

Understanding still has to be earned.

Is this just dependency injection?

Familiar starting point

Abstract individual services.
Substitute implementations.

Additional structure here

Abstract how computations
compose.
Vary the carrier and interpretation.

Use the extra abstraction only when its benefits earn its cost.

Ordinary DI and pure functions may be enough.

What the round-trip does—and does not—say

Required	Established	Not established
A specified language and models. Uniformity across interpretations.	The representation relationship. Recovery up to the stated equality.	Business correctness. Arbitrary rewrite safety. Every future path.

A host-language generic type is not the whole proof.

When generation outruns review

Make the change smaller

Bound the task.
Keep interfaces explicit.
Avoid an unreviewable code dump.

Make the evidence better

Automate suitable checks.
Examine consequential assumptions.
Use independent evidence.

Generating less at once is a legitimate engineering response.

Help colleagues develop judgement

Learning move	Use AI without losing the feedback
Predict	State what the code will do before running it.
Investigate	Trace the failure before accepting the patch.
Compare	Ask for alternatives and examine their assumptions.
Own a bounded change	Implement, observe consequences, and receive review.

Don't replace an apprenticeship with an approval queue.

Further reading

[Froggy Tree House](#)

The small program that introduces interpretation.

[The Algebra of Intent](#)

The original C# architectural example.

[What Mitchell Wand Taught Me About Intent](#)

A public account of the early corrections.

Background reading; some earlier claims were subsequently narrowed.